UNIBALSAS · CENTRO UNIVERSITÁRIO

Workshop · Desenvolvimento de APIs REST e Integração de Sistemas com Laravel

PRIMEIRO ENCONTRO

APIs REST com Laravel

```
GET /api/eventos
POST /api/eventos/1/inscricoes
HTTP/1.1 201 Created
{ "status": "confirmada" }
```

do protocolo HTTP a uma API de inscrições para eventos: rotas, controllers, banco de dados, validação, login com token, regras de acesso por dono e testes de endpoints com o Postman.

Construindo a API: do zero ao primeiro endpoint de inscrição

WORKSHOP
APIs REST e Integração com Laravel

PROFESSOR
Alex Rogaleski Marques

CONTATO
alex.rogaleski@prof.unibalsas.edu.br

Tutorial prático e material de projeção em sala · Workshop em dois encontros

1

ABERTURA

Objetivos, o projeto e o roteiro

Quase todo sistema que usamos no dia a dia conversa com outros sistemas: o aplicativo do banco consulta um servidor, a loja virtual chama a operadora de pagamentos, o sistema acadêmico envia notas para outro órgão. Essa conversa acontece por meio de **APIs**. Neste workshop vamos construir uma do zero: uma **API REST para inscrições em eventos**, escrita em **PHP com o framework Laravel**. Hoje, a API ganha forma: eventos, inscrições, banco de dados, regras de negócio, login e regras de acesso. No próximo encontro, ela passa a conversar com uma plataforma de pagamentos real.

COMO USAR ESTE MATERIAL

- **Siga na ordem.** Cada etapa usa o que foi construído nas anteriores.
- Cada **Passo** traz o código e, logo abaixo, a **resposta real** que a API devolveu quando este material foi preparado. Datas, horas e números de identificação (`id`) da sua execução serão diferentes; o resto deve coincidir.
- Os boxes **"Confira antes de seguir"** são pontos de verificação. Se algo não bater, resolva antes de avançar: um erro no começo vira cinco erros no fim.
- O material continua útil depois do workshop: refaça o projeto em casa, sem pressa.

Objetivos de aprendizagem

Ao final deste encontro, espera-se que o participante seja capaz de:

- Descrever uma requisição e uma resposta HTTP: método, URL, cabeçalhos, corpo e status.
- Criar um projeto Laravel e preparar as rotas de API.
- Implementar as operações de leitura, criação, alteração e remoção, validando os dados de entrada e devolvendo os status adequados.
- Proteger as rotas com cadastro, login e token de acesso (Sanctum).
- Testar e documentar os endpoints com coleções e ambientes do Postman.
- Explicar o estilo REST e projetar as rotas de uma API a partir dos seus recursos.
- Modelar tabelas com migrations e acessá-las com models do Eloquent.
- Relacionar duas tabelas e aplicar regras de negócio a um endpoint.
- Restringir ações ao dono do registro com Policies, distinguindo 401 de 403.

O projeto: inscrições para eventos

A instituição promove palestras, minicursos e workshops. Alguns são gratuitos; outros cobram uma taxa. Precisamos de um sistema que **cadastre os eventos**, **receba inscrições** respeitando o número de vagas e, para os eventos pagos, **cobre a taxa** e **confirme a inscrição quando o pagamento cair**. O sistema não terá telas: será uma API, que poderá ser usada por um site, um aplicativo de celular ou outro sistema. A Figura 1.1 mostra as peças e em que encontro cada uma entra.

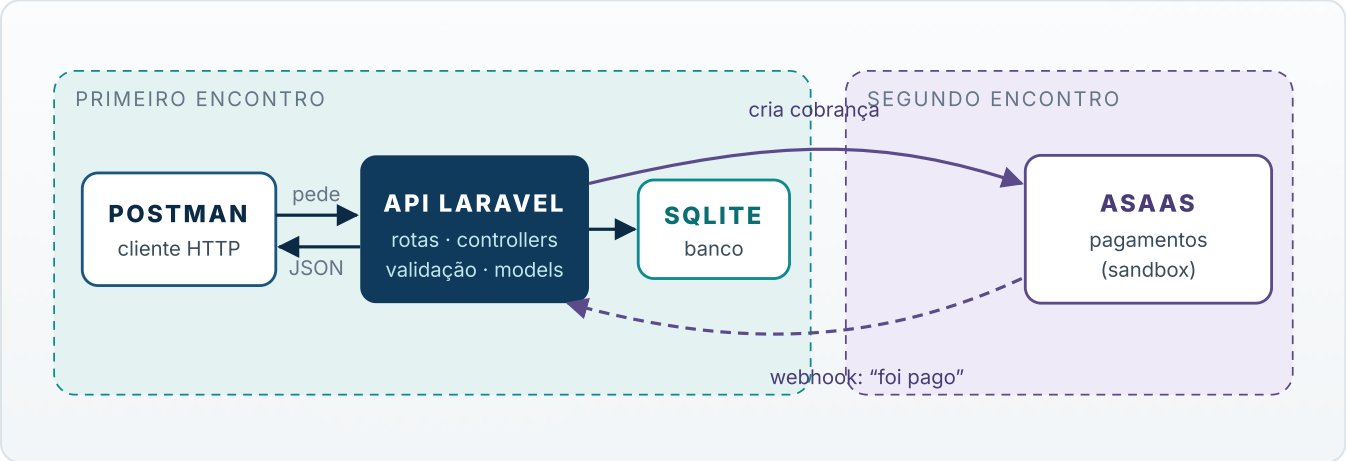


Figura 1.1 — Arquitetura do sistema de inscrições. Hoje construímos a parte da esquerda; no próximo encontro, a integração com a plataforma de pagamentos.

Roteiro do encontro

1	**Como sistemas conversam.** HTTP, REST e JSON; o projeto das rotas da API.
2	**O projeto Laravel.** Criação, estrutura de pastas e a primeira rota.
3	**Banco de dados.** Migration, model e dados de exemplo para os eventos.
4	**Controller e rotas.** Listar e detalhar eventos; ambiente e coleção no Postman.
5	**Escrita e validação.** Criar, alterar e remover eventos; erros 422.
6	**Inscrições.** Relacionamento entre tabelas e regras de negócio.
7	**Autenticação.** Cadastro, login, token e rotas protegidas.
8	**Autorização.** O dono de cada registro, Policies e o 403.
9	**Síntese técnica.** O mapa da API e os erros mais comuns.
10	**Exercícios.** Seis exercícios progressivos e um desafio.

Os exemplos foram executados com **PHP 8.5**, **Laravel 13.39** e **SQLite**. As respostas mostradas são as reais, copiadas da execução.

2

ETAPA 1 · FUNDAMENTOS

Como sistemas conversam: HTTP, REST e JSON

DEFINIÇÃO — API

Uma **API** (*Application Programming Interface*) é um conjunto de operações que um sistema oferece para ser usado **por outros programas**, e não por pessoas. Uma **API web** expõe essas operações por meio do protocolo **HTTP**: cada operação é um endereço que recebe uma requisição e devolve uma resposta.

A requisição e a resposta

O HTTP é um protocolo de **pedido e resposta**. O cliente envia uma **requisição** com quatro partes: o **método** (o que se quer fazer), a **URL** (sobre qual recurso), os **cabeçalhos** (informações sobre o pedido, como o formato aceito ou uma credencial) e, quando faz sentido, um **corpo** com dados. O servidor devolve uma **resposta** com um **código de status**, cabeçalhos e um corpo. A Figura 2.1 mostra uma troca real da API que vamos construir.

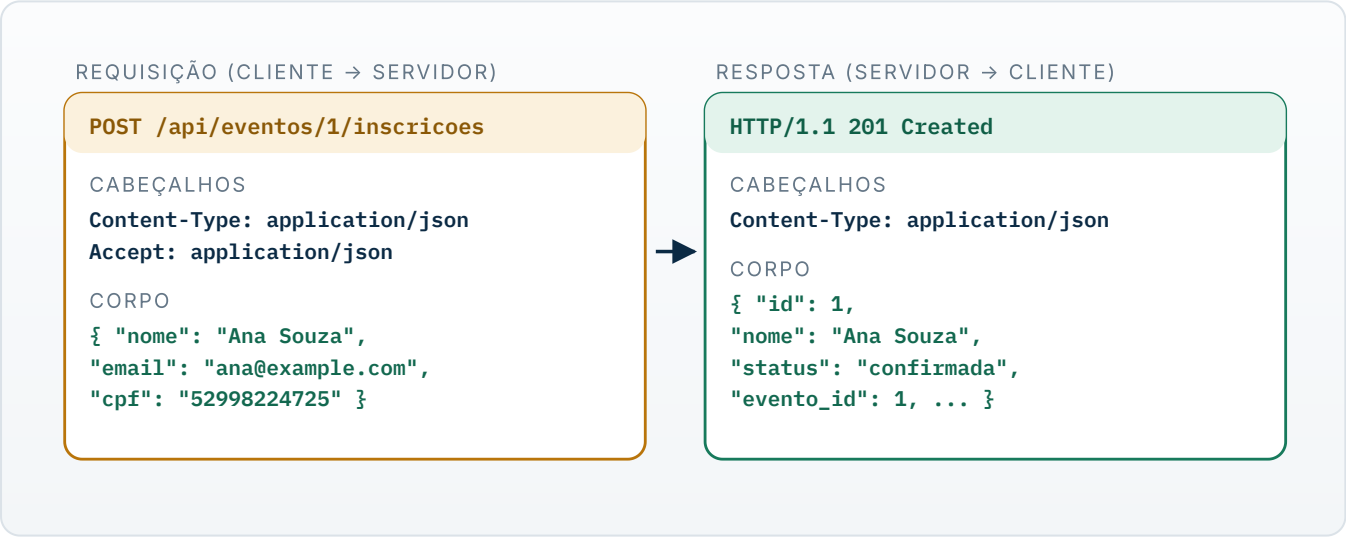


Figura 2.1 — Anatomia de uma troca HTTP: inscrição de uma participante no evento 1.

O **JSON** (*JavaScript Object Notation*) é o formato de dados que as APIs modernas usam no corpo. Ele tem apenas objetos (`{ }`, pares de chave e valor), listas (`[ ]`), textos entre aspas duplas, números, `true` / `false` e `null`. É legível por pessoas e por praticamente qualquer linguagem de programação, e é por isso que virou a língua comum entre sistemas.

Métodos e códigos de status

O método diz **o que fazer**; o código de status diz **como terminou**. A primeira casa do código já informa muito: `2xx` é sucesso, `4xx` é erro de quem pediu e `5xx` é erro do servidor. Os Quadros

2.1 e 2.2 trazem os que usaremos no workshop.

QUADRO 2.1 — MÉTODOS HTTP

Método	Intenção
`GET`	Buscar um recurso ou uma lista. Não altera nada.
`POST`	Criar um recurso novo.
`PUT`	Substituir um recurso inteiro.
`PATCH`	Alterar parte de um recurso.
`DELETE`	Remover um recurso.

QUADRO 2.2 — CÓDIGOS DE STATUS

Código	Significado
`200`	OK: deu certo e há corpo na resposta.
`201`	Created: o recurso foi criado.
`204`	No Content: deu certo, sem corpo.
`401`	Unauthorized: falta um token válido.
`404`	Not Found: o recurso não existe.
`409`	Conflict: viola uma regra (ex.: sem vagas).
`422`	Unprocessable: dados inválidos.
`502`	Bad Gateway: um serviço externo falhou.

O estilo REST

DEFINIÇÃO — REST

REST (*Representational State Transfer*) é um estilo de projetar APIs web. As ideias centrais são: organizar a API em torno de **recursos** (substantivos, como *eventos* e *inscrições*), identificar cada recurso por uma **URL**, usar os **métodos HTTP** para dizer a operação e não guardar estado de conversa no servidor: cada requisição traz tudo o que é preciso para ser atendida.

Na prática, isso significa que a URL não carrega verbos. Em vez de `/criarEvento` e `/apagarEvento?id=3`, há um único endereço por recurso, e o método faz a diferença. O Quadro 2.3 é o **projeto da API** que vamos implementar hoje. Vale olhar para ele com atenção: ao final do encontro, cada linha será um endpoint funcionando.

QUADRO 2.3 — PROJETO DAS ROTAS DA API DE INSCRIÇÕES

Método	URL	O que faz	Sucesso
`GET`	/api/eventos	Lista os eventos	200
`POST`	/api/eventos	Cria um evento	201
`GET`	/api/eventos/{id}	Detalha um evento	200
`PATCH`	/api/eventos/{id}	Altera um evento	200
`DELETE`	/api/eventos/{id}	Remove um evento	204
`GET`	/api/eventos/{id}/inscricoes	Lista as inscrições de um evento	200
`POST`	/api/eventos/{id}/inscricoes	Inscreve uma pessoa no evento	201
`GET`	/api/inscricoes/{id}	Consulta uma inscrição	200

Na Etapa 7, a API ganha também as rotas de conta (`/api/register` , `/api/login` , `/api/logout` e `/api/user`), e as oito rotas do quadro passam a exigir login. Repare na URL `/api/eventos/{id}/inscricoes` : a inscrição **pertence** a um evento, e a URL mostra essa hierarquia. É o que se chama de **recurso aninhado**.

PARA DISCUSSÃO EM SALA

1. Por que `GET /api/apagar-evento/3` é uma má ideia, mesmo que funcione? Pense em um buscador que segue todos os links que encontra.
2. Uma inscrição recusada por falta de vagas é um erro `4xx` ou `5xx` ? E se o banco de dados estiver fora do ar?
3. Que outros recursos um sistema de eventos real poderia ter? Como seriam as URLs deles?

3

ETAPA 2 · O PROJETO LARAVEL

Criação, estrutura e a primeira rota

DEFINIÇÃO — FRAMEWORK

Um **framework** é uma estrutura pronta que resolve os problemas comuns de um tipo de aplicação, para que o desenvolvedor escreva apenas o que é específico do seu sistema. O **Laravel** é o framework PHP mais usado no mundo: ele já sabe receber requisições HTTP, decidir qual código atende cada URL, conversar com o banco de dados, validar dados e montar respostas JSON.

PASSO 2.1 — CRIAR O PROJETO

```
PS> cd ~\Herd
PS> laravel new inscricoes --database=sqlite --phpunit --no-boost --no-node
PS> cd inscricoes
PS> code .
```

As opções respondem de antemão às perguntas do instalador. Se ele perguntar por um *starter kit*, escolha **None**: não usaremos telas. Ao final, o projeto já tem o banco SQLite criado e as tabelas iniciais do Laravel prontas. O último comando abre a pasta no VS Code; daqui em diante, use o **terminal integrado** dele (Ctrl + ').

A estrutura de pastas

Um projeto Laravel tem muitas pastas, mas neste workshop trabalharemos em poucas. As marcadas abaixo são as que vamos tocar:

```
inscricoes/
├── app/
│   ├── Http/Controllers/   ← os controllers: recebem a requisição e devolvem a resposta
│   └── Models/             ← os models: uma classe por tabela
├── bootstrap/
├── config/                 ← configurações (fuso horário, serviços externos)
├── database/
│   ├── database.sqlite      ← o banco de dados, um único arquivo
│   ├── migrations/         ← a estrutura das tabelas, em código
│   └── seeders/            ← dados de exemplo
├── public/
├── routes/                 ← quais URLs existem e quem as atende
├── storage/                ← logs e arquivos gerados
├── vendor/                 ← bibliotecas baixadas pelo Composer (não mexa)
├── .env                    ← configurações locais e segredos (fora do Git)
└── artisan                 ← a ferramenta de linha de comando do Laravel
```

PASSO 2.2 — ACERTAR O FUSO HORÁRIO

O Laravel vem configurado para o horário universal (UTC). Em `config/app.php`, localize a linha do `timezone` e troque o valor:

```
'timezone' => 'America/Sao_Paulo',
```

PASSO 2.3 — PREPARAR O PROJETO PARA SER UMA API

Um projeto novo só tem rotas para páginas web. O comando `install:api` cria o arquivo de rotas de API, `routes/api.php`, e instala o pacote de autenticação por token (Sanctum), que usaremos na Etapa 7, para o login. Se o comando perguntar se deve executar as migrations, responda **yes**.

```
PS> php artisan install:api
```

TERMINAL

```
… (download do pacote laravel/sanctum pelo Composer) …

   INFO  Published API routes file.

   INFO  Running migrations.

  2026_11_02_174114_create_personal_access_tokens_table ....................... 36.22ms
DONE

   INFO  API scaffolding installed. Please add the [Laravel\Sanctum\HasApiTokens] trait to
your User model.
```

Toda rota escrita em `routes/api.php` ganha automaticamente o prefixo `/api` na URL. O aviso sobre o `HasApiTokens` fica para a Etapa 7, quando a API ganhar login.

PASSO 2.4 — A PRIMEIRA ROTA

Abra `routes/api.php` e substitua **todo** o conteúdo por:

routes/api.php

```php
<?php

use Illuminate\Support\Facades\Route;

Route::get('/ping', function () {
    return ['mensagem' => 'API de inscrições no ar!', 'hora' => now()->format('H:i:s')];
});
```

Uma rota liga um **método** e uma **URL** a um trecho de código. Aqui, um `GET` em `/api/ping` executa a função, que devolve um *array* PHP. O Laravel converte o array em JSON sozinho. Agora suba o servidor de desenvolvimento, que ficará rodando nesse terminal; para os próximos comandos, abra um **segundo terminal** (botão **+** do painel).

```
PS> php artisan serve
```

PASSO 2.5 — O PRIMEIRO TESTE NO POSTMAN

GET	http://127.0.0.1:8000/api/ping	200 OK

RESPOSTA

```
{
    "mensagem": "API de inscrições no ar!",
    "hora": "17:41:16"
}
```

CONFIRA ANTES DE SEGUIR

O Postman mostrou `200 OK` e a hora local, não a de Greenwich. Se a hora vier três horas adiantada, o Passo 2.2 não foi salvo. Se der *Could not send request*, o `php artisan serve` não está rodando.

Organizando o Postman: ambiente e coleção

Vamos escrever a URL base dezenas de vezes. Em vez de repeti-la, crie uma **variável de ambiente**:

1. No Postman, menu lateral **Environments → +**. Nome: `Workshop local`.
2. Crie a variável `base_url` com o valor `http://127.0.0.1:8000/api` e salve.
3. No canto superior direito, selecione o ambiente `Workshop local`.
4. Em **Collections → +**, crie a coleção `API de Inscrições`. Salve nela cada requisição que fizer daqui em diante (Ctrl + S), com um nome descritivo, como *Eventos — listar*.

A partir de agora, as URLs serão escritas como `{{base_url}}/ping`. Se um dia a API mudar de endereço, basta trocar a variável. Ao final do workshop, a coleção será a **documentação executável** da sua API.

ETAPA 3 · PERSISTÊNCIA

4 Migration, model e dados de exemplo

A API precisa guardar os eventos em algum lugar. No Laravel, três peças trabalham juntas: a **migration** descreve a tabela, o **model** representa a tabela no código PHP e o **seeder** preenche a tabela com dados de exemplo. Um único comando cria as três:

PASSO 3.1 — GERAR MODEL, MIGRATION E SEEDER

```
PS> php artisan make:model Evento -ms
```

```
TERMINAL
   INFO  Model [app/Models/Evento.php] created successfully.

   INFO  Migration [database/migrations/2026_11_02_174116_create_eventos_table.php] created successfully.

   INFO  Seeder [database/seeders/EventoSeeder.php] created successfully.
```

A opção `-m` pede a migration e a `-s`, o seeder. Repare que o Laravel chamou a tabela de `eventos`: por convenção, o model fica no singular e a tabela, no plural.

DEFINIÇÃO — MIGRATION

Uma **migration** é a estrutura de uma tabela escrita em código, com data no nome do arquivo. Em vez de cada desenvolvedor criar tabelas à mão, o projeto carrega as migrations, e o comando `php artisan migrate` aplica, em ordem, as que ainda não foram executadas. É o "controle de versão" do banco.

PASSO 3.2 — AS COLUNAS DO EVENTO

Abra a migration recém-criada em `database/migrations/` (o nome termina em `_create_eventos_table.php`) e complete o método `up()`:

database/migrations/…_create_eventos_table.php (trecho)

```php
    public function up(): void
    {
        Schema::create('eventos', function (Blueprint $table) {
            $table->id();
            $table->string('titulo');
            $table->text('descricao')->nullable();
            $table->string('local');
            $table->dateTime('data');
            $table->unsignedInteger('vagas');
            $table->decimal('valor', 8, 2)->default(0);
            $table->timestamps();
        });
    }
```

Cada linha é uma coluna: `id()` cria a chave primária autoincrementada; `nullable()` permite deixar a descrição vazia; `decimal('valor', 8, 2)` guarda dinheiro com duas casas, sem os erros de arredondamento de um número de ponto flutuante; e `timestamps()` cria `created_at` e `updated_at`, que o Laravel preenche sozinho. Evento gratuito é evento com valor zero.

PASSO 3.3 — O MODEL

app/Models/Evento.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Evento extends Model
{
    protected $fillable = ['titulo', 'descricao', 'local', 'data', 'vagas', 'valor'];

    protected function casts(): array
    {
        return [
            'data' => 'datetime:Y-m-d H:i',
            'vagas' => 'integer',
            'valor' => 'decimal:2',
        ];
    }
}
```

O model `Evento` já sabe, pelo nome, que sua tabela é `eventos`. Duas declarações completam a classe. A lista `$fillable` diz quais colunas podem ser preenchidas em massa, a partir dos dados de uma requisição; é uma proteção: sem ela, alguém poderia enviar, por exemplo, um `id` ou um `created_at` e o Laravel gravaria. O método `casts()` diz como converter as colunas ao ler e ao gerar JSON: a data no formato `AAAA-MM-DD HH:MM`, as vagas como inteiro e o valor com duas casas.

PASSO 3.4 — DADOS DE EXEMPLO

database/seeders/EventoSeeder.php

```php
<?php

namespace Database\Seeders;

use App\Models\Evento;
use Illuminate\Database\Seeder;

class EventoSeeder extends Seeder
{
    public function run(): void
    {
        Evento::create([
            'titulo' => 'Workshop de APIs com Laravel',
            'descricao' => 'APIs REST, integração e webhooks na prática.',
            'local' => 'Laboratório 3',
            'data' => '2026-12-05 19:00',
            'vagas' => 30,
            'valor' => 0,
        ]);

        Evento::create([
            'titulo' => 'Minicurso de Git e GitHub',
            'descricao' => 'Versionamento de código do zero ao pull request.',
            'local' => 'Auditório',
            'data' => '2026-12-12 14:00',
            'vagas' => 2,
            'valor' => 25.00,
        ]);
    }
}
```

O segundo evento tem **só duas vagas** de propósito: assim poderemos testar a regra de lotação sem precisar de trinta inscrições. Para que o seeder rode, registre-o no seeder principal:

database/seeders/DatabaseSeeder.php

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        $this->call(EventoSeeder::class);
    }
}
```

PASSO 3.5 — CRIAR A TABELA E POPULAR

```
PS> php artisan migrate
PS> php artisan db:seed
```

TERMINAL

```
   INFO  Running migrations.

  2026_11_02_174116_create_eventos_table ..................................... 13.18ms
DONE

   INFO  Seeding database.

  Database\Seeders\EventoSeeder ..............................................
RUNNING
  Database\Seeders\EventoSeeder ............................................... 44 ms
DONE
```

CONFIRA ANTES DE SEGUIR

No VS Code, abra `database/database.sqlite`: a tabela `eventos` tem duas linhas. Errou alguma coluna? O comando `php artisan migrate:fresh --seed` **apaga todas as tabelas**, refaz as migrations e roda os seeders. Em desenvolvimento, é o jeito mais rápido de recomeçar; em produção, jamais.

ETAPA 4 · CONTROLLER E ROTAS

5 Listar e detalhar eventos

DEFINIÇÃO — CONTROLLER

Um **controller** é a classe que **atende as requisições** de um recurso. Cada método público é uma **ação**: recebe os dados da requisição, usa os models para ler ou gravar no banco e devolve a resposta. A rota decide *quem* atende; o controller decide *o que* fazer.



o Laravel converte o model em JSON e monta a resposta HTTP

Figura 5.1 — O caminho de uma requisição dentro do Laravel.

PASSO 4.1 — GERAR O CONTROLLER

```
PS> php artisan make:controller EventoController --api --model=Evento
```

TERMINAL

```
   INFO  Controller [app/Http/Controllers/EventoController.php] created successfully.
```

A opção `--api` gera os cinco métodos de um recurso REST (`index`, `store`, `show`, `update` e `destroy`), vazios. Vamos implementar primeiro os dois de leitura. Substitua o conteúdo do arquivo por:

app/Http/Controllers/EventoController.php (primeira versão)

```php
<?php

namespace App\Http\Controllers;

use App\Models\Evento;
use Illuminate\Http\Request;

class EventoController extends Controller
{
    public function index()
    {
        return Evento::orderBy('data')->get();
    }

    public function store(Request $request)
    {
        //
    }

    public function show(Evento $evento)
    {
        return $evento;
    }

    public function update(Request $request, Evento $evento)
    {
        //
    }

    public function destroy(Evento $evento)
    {
        //
    }
}
```

O `index` busca todos os eventos ordenados pela data. O `show` merece atenção: ele recebe um `Evento $evento` já pronto. É o **route model binding**: o Laravel lê o número da URL, busca o evento no banco e entrega o objeto ao método. Se o evento não existir, responde `404` antes mesmo de o método rodar.

PASSO 4.2 — REGISTRAR AS ROTAS DO RECURSO

routes/api.php

```php
<?php

use App\Http\Controllers\EventoController;
use Illuminate\Support\Facades\Route;

Route::get('/ping', function () {
    return ['mensagem' => 'API de inscrições no ar!', 'hora' => now()->format('H:i:s')];
});

Route::apiResource('eventos', EventoController::class);
```

Uma única linha, `Route::apiResource`, cria as cinco rotas do Quadro 2.3 para eventos. O comando `route:list` mostra o que foi registrado:

```
PS> php artisan route:list --path=api
```

TERMINAL

```
GET|HEAD        api/eventos ..... eventos.index › EventoController@index
POST            api/eventos ..... eventos.store › EventoController@store
GET|HEAD        api/eventos/{evento} ..... eventos.show › EventoController@show
PUT|PATCH       api/eventos/{evento} .. eventos.update › EventoController@update
DELETE          api/eventos/{evento} eventos.destroy › EventoController@destroy
GET|HEAD        api/ping ..... routes/api.php:6

                                                  Showing [6] routes
```

PASSO 4.3 — TESTAR A LEITURA

GET {{base_url}}/eventos 200 OK

RESPOSTA


```
[
    {
        "id": 1,
        "titulo": "Workshop de APIs com Laravel",
        "descricao": "APIs REST, integração e webhooks na prática.",
        "local": "Laboratório 3",
        "data": "2026-12-05 19:00",
        "vagas": 30,
        "valor": "0.00",
        "created_at": "2026-11-02T20:41:17.000000Z",
        "updated_at": "2026-11-02T20:41:17.000000Z"
    },
    {
        "id": 2,
        "titulo": "Minicurso de Git e GitHub",
        "descricao": "Versionamento de código do zero ao pull request.",
        "local": "Auditório",
        "data": "2026-12-12 14:00",
        "vagas": 2,
        "valor": "25.00",
        "created_at": "2026-11-02T20:41:18.000000Z",
        "updated_at": "2026-11-02T20:41:18.000000Z"
    }
]
```


A lista é um **array JSON** de objetos, um por evento. Agora um evento só, e depois um que não existe:

GET {{base_url}}/eventos/1 200 OK

RESPOSTA


```
{
    "id": 1,
    "titulo": "Workshop de APIs com Laravel",
    "descricao": "APIs REST, integração e webhooks na prática.",
    "local": "Laboratório 3",
    "data": "2026-12-05 19:00",
    "vagas": 30,
    "valor": "0.00",
    "created_at": "2026-11-02T20:41:17.000000Z",
    "updated_at": "2026-11-02T20:41:17.000000Z"
}
```

GET {{base_url}}/eventos/99 404 Not Found

RESPOSTA

```
{
    "message": "No query results for model [App\\Models\\Evento] 99",
    "exception": "Symfony\\Component\\HttpKernel\\Exception\\NotFoundHttpException",
    "file": "…",
    "line": 773,
    "trace": [ … ]
}
```

Ninguém escreveu código para o caso do evento inexistente, e mesmo assim a API respondeu corretamente com `404`. O corpo traz detalhes técnicos (arquivo, linha e a pilha de chamadas, `trace`, aqui resumida) porque o projeto está em modo de depuração, com `APP_DEBUG=true` no `.env`. Em produção, essa variável fica `false`, e o cliente recebe só a mensagem.

PARA DISCUSSÃO EM SALA

1. Por que o `valor` aparece como texto (`"25.00"`) e as `vagas` como número? Qual a vantagem de o dinheiro não virar ponto flutuante?
2. Mostrar o caminho dos arquivos do servidor numa resposta de erro é um risco. Que informação um atacante tiraria dali?
3. Se a lista tivesse dez mil eventos, o que aconteceria com o `index`? Como resolver?

ETAPA 5 · ESCRITA E VALIDAÇÃO

6 Criar, alterar e remover eventos

Uma API que só lê não serve para muito. Os três métodos que faltam recebem dados do cliente, e aqui mora a regra mais importante do desenvolvimento de APIs: **nunca confie no que chega.** Antes de gravar, todo dado é **validado**.

DEFINIÇÃO — VALIDAÇÃO

Validar é conferir se os dados recebidos obedecem às regras do domínio antes de usá-los: campos obrigatórios presentes, tipos corretos, limites respeitados. No Laravel, o método `$request->validate()` recebe as regras de cada campo; se alguma falhar, ele **interrompe** a ação e responde `422` com a lista de erros, sem que nenhuma linha seguinte seja executada.

PASSO 5.1 — O CONTROLLER COMPLETO

Substitua os três métodos vazios. O arquivo final fica assim:

app/Http/Controllers/EventoController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Evento;
use Illuminate\Http\Request;

class EventoController extends Controller
{
    public function index()
    {
        return Evento::orderBy('data')->get();
    }

    public function store(Request $request)
    {
        $dados = $request->validate([
            'titulo' => 'required|string|max:120',
            'descricao' => 'nullable|string',
            'local' => 'required|string|max:120',
            'data' => 'required|date|after:now',
            'vagas' => 'required|integer|min:1',
            'valor' => 'required|numeric|min:0',
        ]);

        $evento = Evento::create($dados);

        return response()->json($evento, 201);
    }

    public function show(Evento $evento)
    {
        return $evento;
    }

    public function update(Request $request, Evento $evento)
    {
        $dados = $request->validate([
            'titulo' => 'sometimes|required|string|max:120',
            'descricao' => 'nullable|string',
            'local' => 'sometimes|required|string|max:120',
            'data' => 'sometimes|required|date|after:now',
            'vagas' => 'sometimes|required|integer|min:1',
            'valor' => 'sometimes|required|numeric|min:0',
        ]);

        $evento->update($dados);

        return $evento;
    }

    public function destroy(Evento $evento)
    {
        $evento->delete();
```

```
        return response()->noContent();
    }
}
```

As regras são lidas da esquerda para a direita, separadas por `|`. No `store`, `'data' => 'required|date|after:now'` exige a data, exige que seja uma data e exige que esteja no futuro. O método devolve só os campos validados, e é essa lista (`$dados`), e não a requisição inteira, que vai para o `create`. No `update`, a regra `sometimes` diz "valide só se o campo vier": é o que permite alterar só as vagas sem reenviar o evento inteiro, como pede o método `PATCH`. O `destroy` devolve `204`, sem corpo.

PASSO 5.2 — CRIAR UM EVENTO

No Postman: método `POST`, aba **Body → raw → JSON**. Na aba **Headers**, acrescente `Accept: application/json`.

POST {{base_url}}/eventos — **201 Created**

```
{
    "titulo": "Hackathon StartUB",
    "local": "Bloco B",
    "data": "2026-12-20 08:00",
    "vagas": 50,
    "valor": 0
}
```

RESPOSTA

```
{
    "titulo": "Hackathon StartUB",
    "local": "Bloco B",
    "data": "2026-12-20 08:00",
    "vagas": 50,
    "valor": "0.00",
    "updated_at": "2026-11-02T20:41:20.000000Z",
    "created_at": "2026-11-02T20:41:20.000000Z",
    "id": 3
}
```

Se ao reproduzir o passo a data já tiver passado, use uma data futura: a regra `after:now` recusará a anterior. O `201` e o `id` gerado confirmam a criação.

PASSO 5.3 — PROVOCAR ERROS DE VALIDAÇÃO

POST {{base_url}}/eventos — **422 Unprocessable Content**

```
{ "titulo": "", "data": "ontem", "vagas": 0 }
```

RESPOSTA

```
{
    "message": "The titulo field is required. (and 5 more errors)",
    "errors": {
        "titulo": [
            "The titulo field is required."
        ],
        "local": [
            "The local field is required."
        ],
        "data": [
            "The data field must be a valid date.",
            "The data field must be a date after now."
        ],
        "vagas": [
            "The vagas field must be at least 1."
        ],
        "valor": [
            "The valor field is required."
        ]
    }
}
```

Cinco campos, seis problemas, todos relatados de uma vez, cada um sob o nome do seu campo. É exatamente o formato de que um front-end precisa para pintar de vermelho cada campo errado do formulário. As mensagens vêm em inglês, o idioma padrão do Laravel; traduzi-las é um dos exercícios da Seção 11.

POR QUE O CABEÇALHO ACCEPT?

O `Accept: application/json` avisa o servidor de que o cliente quer JSON. Num projeto Laravel 13, as rotas que começam com `/api` já respondem em JSON mesmo sem ele, mas em versões anteriores, ou em rotas fora de `/api`, um erro de validação sem esse cabeçalho vira um *redirecionamento* para uma página HTML. Enviar o `Accept` é o hábito correto em qualquer API.

PASSO 5.4 — ALTERAR E REMOVER

PATCH {{base_url}}/eventos/3 200 OK

```
{ "vagas": 60 }
```

RESPOSTA

```
{
    "id": 3,
    "titulo": "Hackathon StartUB",
    "descricao": null,
    "local": "Bloco B",
    "data": "2026-12-20 08:00",
    "vagas": 60,
    "valor": "0.00",
    "created_at": "2026-11-02T20:41:20.000000Z",
    "updated_at": "2026-11-02T20:41:20.000000Z"
}
```

DELETE	{{base_url}}/eventos/3	204 No Content

A resposta não tem corpo. Repita o `GET {{base_url}}/eventos/3`: agora ele devolve `404`.

CONFIRA ANTES DE SEGUIR

A coleção do Postman tem sete requisições salvas: ping, listar, detalhar, criar, criar com erro, alterar e remover. O evento 3 foi removido, e os eventos 1 e 2 continuam lá.

7

ETAPA 6 · INSCRIÇÕES

Relacionamento e regras de negócio

Agora o recurso principal do sistema. Uma inscrição tem nome, e-mail e CPF de quem se inscreve, um **status** e pertence a **um** evento; um evento tem **muitas** inscrições. E há regras: não se aceita inscrição sem vaga, nem duas inscrições com o mesmo e-mail no mesmo evento, e a inscrição em evento gratuito já nasce confirmada, enquanto a de evento pago nasce **pendente**, aguardando pagamento.

PASSO 6.1 — MODEL E MIGRATION, COM UM CUIDADO

```
PS> php artisan make:model Inscricao
PS> php artisan make:migration create_inscricoes_table
```

TERMINAL

```
   INFO  Model [app/Models/Inscricao.php] created successfully.

   INFO  Migration [database/migrations/2026_11_02_174122_create_inscricoes_table.php]
created successfully.
```

O LARAVEL PLURALIZA EM INGLÊS

Por que não usamos `make:model Inscricao -m`, como fizemos com o evento? Porque as regras de plural do Laravel são as do inglês: *evento* vira *eventos* por sorte, mas *inscricao* viraria `inscricaos`. Por isso criamos a migration com o nome certo à mão e, no model, informamos o nome da tabela com `protected $table = 'inscricoes'`.

PASSO 6.2 — A TABELA DE INSCRIÇÕES

database/migrations/…_create_inscricoes_table.php (trecho)

```php
        Schema::create('inscricoes', function (Blueprint $table) {
            $table->id();
            $table->foreignId('evento_id')->constrained()->cascadeOnDelete();
            $table->string('nome');
            $table->string('email');
            $table->string('cpf', 11);
            $table->string('status')->default('pendente');
            $table->timestamps();

            $table->unique(['evento_id', 'email']);
        });
```

A linha `foreignId('evento_id')->constrained()` cria a **chave estrangeira** para a tabela `eventos`, e `cascadeOnDelete()` apaga as inscrições se o evento for apagado. O `unique` composto garante, **no próprio banco**, que um e-mail só se inscreve uma vez em cada evento: mesmo que um bug passe pela validação, o banco recusa.

PASSO 6.3 — OS DOIS LADOS DO RELACIONAMENTO

app/Models/Inscricao.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Inscricao extends Model
{
    protected $table = 'inscricoes';

    protected $fillable = ['nome', 'email', 'cpf', 'status'];

    public function evento(): BelongsTo
    {
        return $this->belongsTo(Evento::class);
    }
}
```

No `Evento`, acrescente o lado "muitos", com os dois `use` no topo do arquivo:

app/Models/Evento.php (acréscimos)

```php
use Illuminate\Database\Eloquent\Relations\HasMany;

    public function inscricoes(): HasMany
    {
        return $this->hasMany(Inscricao::class);
    }
```

Com isso, `$evento->inscricoes()` passa a significar "as inscrições deste evento", e `$inscricao->evento`, "o evento desta inscrição".

PASSO 6.4 — O CONTROLLER DAS INSCRIÇÕES

```
PS> php artisan migrate
PS> php artisan make:controller InscricaoController
```

app/Http/Controllers/InscricaoController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Evento;
use App\Models\Inscricao;
use Illuminate\Http\Request;
use Illuminate\Validation\Rule;

class InscricaoController extends Controller
{
    public function index(Evento $evento)
    {
        return $evento->inscricoes()->orderBy('nome')->get();
    }

    public function store(Request $request, Evento $evento)
    {
        $dados = $request->validate([
            'nome' => 'required|string|max:120',
            'email' => [
                'required', 'email',
                Rule::unique('inscricoes')->where('evento_id', $evento->id),
            ],
            'cpf' => 'required|digits:11',
        ]);

        if ($evento->inscricoes()->count() >= $evento->vagas) {
            return response()->json(['message' => 'Não há vagas para este evento.'], 409);
        }

        $dados['status'] = $evento->valor > 0 ? 'pendente' : 'confirmada';

        $inscricao = $evento->inscricoes()->create($dados);

        return response()->json($inscricao, 201);
    }

    public function show(Inscricao $inscricao)
    {
        return $inscricao->load('evento');
    }
}
```

O `store` é o coração do sistema e aplica as regras em camadas. Primeiro a **validação**: a regra `Rule::unique('inscricoes')->where('evento_id', …)` recusa e-mail repetido *no mesmo evento*, e `digits:11` exige o CPF só com números. Depois a **regra de negócio**: se o número de inscrições já alcançou as vagas, a resposta é `409 Conflict`: os dados estão corretos, mas a operação conflita com o estado atual do sistema. Por fim, o **status inicial** depende do valor do evento, e `$evento->inscricoes()->create()` grava a inscrição já com o `evento_id` preenchido pelo relacionamento.

PASSO 6.5 — AS ROTAS FINAIS

routes/api.php

```php
<?php

use App\Http\Controllers\EventoController;
use App\Http\Controllers\InscricaoController;
use Illuminate\Support\Facades\Route;

Route::get('/ping', function () {
    return ['mensagem' => 'API de inscrições no ar!', 'hora' => now()->format('H:i:s')];
});

Route::apiResource('eventos', EventoController::class);

Route::get('/eventos/{evento}/inscricoes', [InscricaoController::class, 'index']);
Route::post('/eventos/{evento}/inscricoes', [InscricaoController::class, 'store']);
Route::get('/inscricoes/{inscricao}', [InscricaoController::class, 'show']);
```

TERMINAL

```
GET|HEAD        api/eventos ..... eventos.index › EventoController@index
POST            api/eventos ..... eventos.store › EventoController@store
GET|HEAD        api/eventos/{evento} ..... eventos.show › EventoController@show
PUT|PATCH       api/eventos/{evento} .. eventos.update › EventoController@update
DELETE          api/eventos/{evento} eventos.destroy › EventoController@destroy
GET|HEAD        api/eventos/{evento}/inscricoes ..... InscricaoController@index
POST            api/eventos/{evento}/inscricoes ..... InscricaoController@store
GET|HEAD        api/inscricoes/{inscricao} ..... InscricaoController@show
GET|HEAD        api/ping ..... routes/api.php:7

                                                Showing [9] routes
```

PASSO 6.6 — INSCREVER, REPETIR E LOTAR

Inscrição no evento gratuito (evento 1):

POST {{base_url}}/eventos/1/inscricoes — **201 Created**

```
{ "nome": "Ana Souza", "email": "ana@example.com", "cpf": "52998224725" }
```

RESPOSTA

```
{
    "nome": "Ana Souza",
    "email": "ana@example.com",
    "cpf": "52998224725",
    "status": "confirmada",
    "evento_id": 1,
    "updated_at": "2026-11-02T20:41:23.000000Z",
    "created_at": "2026-11-02T20:41:23.000000Z",
    "id": 1
}
```

A mesma pessoa de novo, agora com o CPF formatado:

POST {{base_url}}/eventos/1/inscricoes — **422 Unprocessable Content**

```
{ "nome": "Ana Souza", "email": "ana@example.com", "cpf": "529.982.247-25" }
```

RESPOSTA


```
{
    "message": "The email has already been taken. (and 1 more error)",
    "errors": {
        "email": [
            "The email has already been taken."
        ],
        "cpf": [
            "The cpf field must be 11 digits."
        ]
    }
}
```


No evento 2, pago e com duas vagas, as duas primeiras inscrições (Bruno e Carla) entram com status `pendente`. A terceira esbarra na regra de lotação:

POST {{base_url}}/eventos/2/inscricoes — **201 Created**

```
{ "nome": "Bruno Lima", "email": "bruno@example.com", "cpf": "11144477735" }
```

RESPOSTA


```
{
    "nome": "Bruno Lima",
    "email": "bruno@example.com",
    "cpf": "11144477735",
    "status": "pendente",
    "evento_id": 2,
    "updated_at": "2026-11-02T20:41:24.000000Z",
    "created_at": "2026-11-02T20:41:24.000000Z",
    "id": 2
}
```


Repita a requisição com `{ "nome": "Carla Dias", "email": "carla@example.com", "cpf": "39053344705" }`: também `201`, também pendente. O evento está lotado.

POST {{base_url}}/eventos/2/inscricoes — **409 Conflict**

```
{ "nome": "Davi Rocha", "email": "davi@example.com", "cpf": "15350946056" }
```

RESPOSTA


```
{
    "message": "Não há vagas para este evento."
}
```

DADOS DE TESTE

Os CPFs deste material são números **fictícios** com dígitos verificadores válidos, e os e-mails usam o domínio `example.com`, reservado internacionalmente para exemplos: nenhuma mensagem enviada para ele chega a alguém. Mantenha esse hábito, principalmente no próximo encontro, quando os dados irão para uma plataforma de pagamentos.

PASSO 6.7 — CONSULTAR UMA INSCRIÇÃO COM O SEU EVENTO

`GET` `{{base_url}}/inscricoes/2` **200 OK**

RESPOSTA


```
{
    "id": 2,
    "evento_id": 2,
    "nome": "Bruno Lima",
    "email": "bruno@example.com",
    "cpf": "11144477735",
    "status": "pendente",
    "created_at": "2026-11-02T20:41:24.000000Z",
    "updated_at": "2026-11-02T20:41:24.000000Z",
    "evento": {
        "id": 2,
        "titulo": "Minicurso de Git e GitHub",
        "descricao": "Versionamento de código do zero ao pull request.",
        "local": "Auditório",
        "data": "2026-12-12 14:00",
        "vagas": 2,
        "valor": "25.00",
        "created_at": "2026-11-02T20:41:18.000000Z",
        "updated_at": "2026-11-02T20:41:18.000000Z"
    }
}
```


O `load('evento')` do método `show` carregou o relacionamento, e o evento veio aninhado dentro da inscrição, numa única resposta.

PARA DISCUSSÃO EM SALA

1. A inscrição de Bruno está `pendente`. Quem deveria mudá-la para `confirmada`, e como esse “alguém” saberia que o pagamento foi feito?
2. Duas pessoas enviam a última inscrição de um evento no mesmo milissegundo. A contagem do `store` protege contra isso? O que mais poderia proteger?
3. O CPF foi recusado por estar formatado. É melhor recusar ou limpar a pontuação antes de validar? Que riscos tem cada escolha?

8

ETAPA 7 · AUTENTICAÇÃO

Cadastro, login e rotas protegidas

Do jeito que está, qualquer pessoa que descubra o endereço da API pode criar, alterar e apagar eventos. Vamos exigir **login**: quem usa a API se cadastra, faz login e recebe um **token**, que passa a enviar em toda requisição. Sem token válido, a resposta é `401`. Ficam abertos só o `/api/ping`, o cadastro e o próprio login.

DEFINIÇÃO — AUTENTICAÇÃO POR TOKEN

Autenticar é descobrir **quem** está fazendo a requisição. Numa API, o jeito mais comum é o **token de acesso**: um texto longo e aleatório que o servidor entrega no login e o cliente devolve em cada requisição, no cabeçalho `Authorization: Bearer <token>`. Como o REST não guarda estado de conversa, é o token que diz, a cada pedido, quem está pedindo. No Laravel, quem emite e confere esses tokens é o **Sanctum**, instalado no Passo 2.3, que os guarda na tabela `personal_access_tokens`.

PASSO 7.1 — O USUÁRIO PODE TER TOKENS

O projeto já tem o model `User` e a tabela `users`. Falta o que o aviso do `install:api` pediu: o *trait* `HasApiTokens`, que dá ao usuário o método `createToken()`.

app/Models/User.php (alterações)

```
use Laravel\Sanctum\HasApiTokens;

    use HasApiTokens, HasFactory, Notifiable;
```

PASSO 7.2 — O CONTROLLER DE AUTENTICAÇÃO

```
PS> php artisan make:controller AuthController
```

TERMINAL

```
   INFO  Controller [app/Http/Controllers/AuthController.php] created successfully.
```

app/Http/Controllers/AuthController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\ValidationException;

class AuthController extends Controller
{
    public function register(Request $request)
    {
        $dados = $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|email|max:255|unique:users',
            'password' => 'required|string|min:8|confirmed',
        ]);

        $user = User::create($dados);
        $token = $user->createToken('front')->plainTextToken;

        return response()->json(['user' => $user, 'token' => $token], 201);
    }

    public function login(Request $request)
    {
        $dados = $request->validate([
            'email' => 'required|email',
            'password' => 'required|string',
        ]);

        $user = User::where('email', $dados['email'])->first();

        if (! $user || ! Hash::check($dados['password'], $user->password)) {
            throw ValidationException::withMessages(['email' => 'Credenciais inválidas.']);
        }

        return ['user' => $user, 'token' => $user->createToken('front')->plainTextToken];
    }

    public function logout(Request $request)
    {
        $request->user()->currentAccessToken()->delete();

        return response()->noContent();
    }
}
```

São três métodos. O `register` valida os dados (a regra `confirmed` exige um campo `password_confirmation` igual à senha), cria o usuário e já devolve um token, com `201`. A senha não é gravada como veio: o model `User` tem um *cast* `'password' => 'hashed'`, que a

transforma num *hash* antes de salvar. O `login` procura o usuário pelo e-mail e confere a senha com `Hash::check`. Repare que e-mail inexistente e senha errada dão a **mesma** mensagem: dizer qual dos dois falhou ajudaria quem tenta adivinhar contas. O `logout` apaga o token usado na requisição, que deixa de valer na hora.

PASSO 7.3 — PROTEGER AS ROTAS

Substitua o conteúdo de `routes/api.php`. As rotas de eventos e inscrições não mudam: elas só passam para dentro de um **grupo** com o *middleware* `auth:sanctum`.

routes/api.php

```php
<?php

use App\Http\Controllers\AuthController;
use App\Http\Controllers\EventoController;
use App\Http\Controllers\InscricaoController;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::get('/ping', function () {
    return ['mensagem' => 'API de inscrições no ar!', 'hora' => now()->format('H:i:s')];
});

Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login'])->middleware('throttle:6,1');

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/user', fn (Request $request) => $request->user());
    Route::post('/logout', [AuthController::class, 'logout']);

    Route::apiResource('eventos', EventoController::class);

    Route::get('/eventos/{evento}/inscricoes', [InscricaoController::class, 'index']);
    Route::post('/eventos/{evento}/inscricoes', [InscricaoController::class, 'store']);
    Route::get('/inscricoes/{inscricao}', [InscricaoController::class, 'show']);
});
```

TERMINAL

```
GET|HEAD        api/eventos ..............  eventos.index › EventoController@index
POST            api/eventos ..............  eventos.store › EventoController@store
GET|HEAD        api/eventos/{evento} ........  eventos.show › EventoController@show
PUT|PATCH       api/eventos/{evento} ....  eventos.update › EventoController@update
DELETE          api/eventos/{evento} ..  eventos.destroy › EventoController@destroy
GET|HEAD        api/eventos/{evento}/inscricoes ........  InscricaoController@index
POST            api/eventos/{evento}/inscricoes ........  InscricaoController@store
GET|HEAD        api/inscricoes/{inscricao} ..............  InscricaoController@show
POST            api/login ..................................  AuthController@login
POST            api/logout .................................  AuthController@logout
GET|HEAD        api/ping ...................................  routes/api.php:9
POST            api/register ............................  AuthController@register
GET|HEAD        api/user ...................................  routes/api.php:17

                                                  Showing [13] routes
```

Um **middleware** é um filtro que roda **antes** do controller. O `auth:sanctum` lê o cabeçalho `Authorization`, procura o token no banco e, se não encontrar, encerra a requisição com `401`, sem que o controller chegue a ser chamado. A rota `/api/user` devolve o usuário dono do token. No login, o `throttle:6,1` limita a **6 tentativas por minuto**; a sétima recebe `429 Too Many Requests`, o que atrapalha quem tenta descobrir senhas por tentativa e erro.

PASSO 7.4 — VISITANTE SEM TOKEN RECEBE 401

Falta um ajuste. O Laravel também serve sites, e num site o visitante sem login é **redirecionado** para a página de login. A nossa API não tem essa página: uma requisição sem token e sem o cabeçalho `Accept` quebraria com erro `500` e a mensagem *Route [login] not defined*. Em `bootstrap/app.php`, troque o comentário vazio do `withMiddleware`:

bootstrap/app.php (trecho)

```
    ->withMiddleware(function (Middleware $middleware): void {
        // API pura, sem tela de login: visitante sem token recebe 401 em vez de redirect.
        $middleware->redirectGuestsTo(fn () => null);
    })
```

PASSO 7.5 — A PORTA FECHADA

Repita no Postman a requisição *Eventos — listar*, que funcionava até agora:

GET {{base_url}}/eventos — 401 Unauthorized

RESPOSTA

```
{
    "message": "Unauthenticated."
}
```

PASSO 7.6 — CADASTRAR E FAZER LOGIN

Crie, na coleção, uma pasta `Auth` com as requisições a seguir. Nas três primeiras (cadastrar, login e o login com erro), abra a aba **Authorization** e escolha **No Auth**: elas não precisam de token.

POST {{base_url}}/register — 201 Created

```
{
    "name": "Paula Mendes",
    "email": "paula@example.com",
    "password": "segredo123",
    "password_confirmation": "segredo123"
}
```

RESPOSTA


```
{
    "user": {
        "name": "Paula Mendes",
        "email": "paula@example.com",
        "updated_at": "2026-11-12T13:10:04.000000Z",
        "created_at": "2026-11-12T13:10:04.000000Z",
        "id": 1
    },
    "token": "1|ql4zGQx7eB0bUu1oXIwaCzBzNex9f9PBXNfzobIOe18c6a83"
}
```


O `token` tem duas partes: o número antes da barra é o `id` do registro em `personal_access_tokens`; o resto é o segredo. A senha não aparece na resposta, porque o model a esconde (`#[Hidden]`). Agora, o login com a senha errada:

POST {{base_url}}/login — **422 Unprocessable Content**

```
{ "email": "paula@example.com", "password": "errada" }
```

RESPOSTA


```
{
    "message": "Credenciais inválidas.",
    "errors": {
        "email": [
            "Credenciais inválidas."
        ]
    }
}
```


O formato é o mesmo dos erros de validação da Etapa 5, com a mensagem sob `errors.email`. E com a senha certa:

POST {{base_url}}/login — **200 OK**

```
{ "email": "paula@example.com", "password": "segredo123" }
```

RESPOSTA


```
{
    "user": {
        "id": 1,
        "name": "Paula Mendes",
        "email": "paula@example.com",
        "email_verified_at": null,
        "created_at": "2026-11-12T13:10:04.000000Z",
        "updated_at": "2026-11-12T13:10:04.000000Z"
    },
    "token": "2|GsOfxvo0hNDRARGnlbx55n2TLRqjlRPSSo8lCSXh257e7a29"
}
```

Cada login gera um token **novo** (este é o de número 2), e os anteriores continuam valendo até alguém fazer logout com eles.

PASSO 7.7 — O POSTMAN GUARDA E ENVIA O TOKEN

Copiar o token à mão a cada login cansa. Vamos deixar o Postman fazer isso:

1. Nas requisições de cadastro e de login, aba **Scripts → Post-response**, escreva a linha abaixo. Depois de cada resposta, ela grava o token na variável `token` do ambiente.

```
pm.environment.set("token", pm.response.json().token);
```

2. Abra as configurações da coleção `API de Inscrições`, aba **Authorization**. Escolha **Bearer Token** e, no campo **Token**, escreva `{{token}}`. Salve.
3. As requisições da coleção vêm com **Inherit auth from parent** e passam a enviar `Authorization: Bearer …` sozinhas. O ping pode ficar como está: a rota é aberta e ignora o cabeçalho.

Faça o login de novo, para o script gravar o token, e repita *Eventos — listar*: agora vem `200` com a lista de sempre. Em seguida, pergunte à API quem é você:

GET `{{base_url}}/user` — **200 OK**

RESPOSTA


```
{
    "id": 1,
    "name": "Paula Mendes",
    "email": "paula@example.com",
    "email_verified_at": null,
    "created_at": "2026-11-12T13:10:04.000000Z",
    "updated_at": "2026-11-12T13:10:04.000000Z"
}
```


PASSO 7.8 — LOGOUT

POST `{{base_url}}/logout` — **204 No Content**

Sem corpo na resposta. O token foi apagado do banco; a mesma requisição de antes, com o mesmo token, agora é recusada:

GET `{{base_url}}/eventos` — **401 Unauthorized**

RESPOSTA


```
{
    "message": "Unauthenticated."
}
```


Faça login mais uma vez antes de seguir: a próxima etapa, os exercícios e o próximo encontro usam a API autenticada.

E QUEM CONSOME A API?

Um front-end web ou um aplicativo segue o mesmo roteiro do Postman: no login ou no cadastro, guarda o `token`; em toda chamada, envia `Accept: application/json` e `Authorization: Bearer <token>`; ao receber `401`, apaga o token e volta para a tela de login; no logout, chama `POST /api/logout` e só então apaga o token. Guardar o token no `localStorage` do navegador é o caminho mais simples, mas um script malicioso injetado na página (XSS) consegue lê-lo. Em produção, vale considerar um cookie `httpOnly`, que o Sanctum também oferece no modo SPA.

CONFIRA ANTES DE SEGUIR

Sem token, `GET {{base_url}}/eventos` responde `401`; depois do login, `200`. A coleção tem a pasta `Auth` com cadastrar, login, login com erro, usuário e logout, e a coleção está configurada com **Bearer Token** `{{token}}`. Se tudo der `401` mesmo depois do login, confira se o script gravou a variável (ícone de olho, ao lado do ambiente) e se o ambiente *Workshop local* está selecionado.

PARA DISCUSSÃO EM SALA

1. O login errado responde `422`, e a rota sem token, `401`. Por que não usar o mesmo código para os dois casos?
2. Qualquer pessoa pode se cadastrar e, logada, apagar o evento de outra pessoa. Autenticar resolveu o problema? Que informação falta no banco para a API saber quem pode alterar cada evento?
3. O token que criamos nunca expira. Que riscos isso traz? Como você limitaria a validade dele?

9

ETAPA 8 · AUTORIZAÇÃO

Cada conta mexe no que é seu

O login resolveu metade do problema: a API sabe **quem** está pedindo, mas ainda deixa qualquer conta alterar e apagar o evento de outra pessoa. Falta decidir **o que** cada um pode fazer. A regra do sistema é a de uma vitrine: todos os eventos ficam visíveis para quem está logado, e qualquer conta pode criar eventos e se inscrever em qualquer um. Já alterar e remover um evento, ou ver a lista de inscritos, é coisa do **dono** do evento. E cada inscrição só pode ser consultada por quem a fez.

DEFINIÇÃO — AUTENTICAÇÃO × AUTORIZAÇÃO

Autenticar é descobrir quem está pedindo (Etapa 7). **Autorizar** é decidir se essa pessoa pode fazer o que está pedindo. Cada falha tem o seu código: `401 Unauthorized` quer dizer "não sei quem é você" (sem token, ou com token inválido), e o cliente resolve fazendo login; `403 Forbidden` quer dizer "sei quem é você, e você não pode fazer isto", e fazer login de novo não adianta. No Laravel, as regras de autorização de um model ficam numa classe própria, a **Policy**.

PASSO 8.1 — QUEM É O DONO: A COLUNA USER_ID

Para saber de quem é cada evento e cada inscrição, as duas tabelas ganham uma chave estrangeira para `users`. Como as tabelas já existem, são duas migrations de alteração:

```
PS> php artisan make:migration add_user_id_to_eventos_table
PS> php artisan make:migration add_user_id_to_inscricoes_table
```

TERMINAL

```
   INFO  Migration [database/migrations/2026_11_12_131849_add_user_id_to_eventos_table.php]
created successfully.

   INFO  Migration
[database/migrations/2026_11_12_131850_add_user_id_to_inscricoes_table.php] created
successfully.
```

database/migrations/…_add_user_id_to_eventos_table.php (trecho)

```php
    public function up(): void
    {
        Schema::table('eventos', function (Blueprint $table) {
            // nullable: eventos criados antes desta versão ficam sem dono
            $table->foreignId('user_id')->nullable()->after('id')
                ->constrained()->cascadeOnDelete();
        });
    }

    public function down(): void
    {
        Schema::table('eventos', function (Blueprint $table) {
            $table->dropConstrainedForeignId('user_id');
        });
    }
```

Na migration das inscrições, o conteúdo é o mesmo, trocando `'eventos'` por `'inscricoes'` e `after('id')` por `after('evento_id')`. Rode `php artisan migrate`.

Três detalhes. A coluna é `nullable` porque os eventos 1 e 2 e as três inscrições já gravadas não têm dono, e o banco não aceitaria uma coluna obrigatória vazia nelas; além disso, nem tudo acontece com alguém logado: o `EventoSeeder` cria eventos sem usuário, e no próximo encontro

quem altera inscrições é o webhook do Asaas, que também não tem usuário. O `cascadeOnDelete()` apaga os eventos e as inscrições de um usuário removido. E o `after()` só tem efeito no MySQL: o SQLite acrescenta a coluna nova no fim da tabela, e é por isso que o `user_id` aparece no fim das respostas lidas do banco. O `down()` usa `dropConstrainedForeignId`, que apaga a chave estrangeira e a coluna de uma vez.

PASSO 8.2 — OS MODELS

No `User`, o lado "muitos" do relacionamento, como fizemos no evento com as inscrições:

app/Models/User.php (acréscimos)

```php
use Illuminate\Database\Eloquent\Relations\HasMany;

    public function eventos(): HasMany
    {
        return $this->hasMany(Evento::class);
    }
```

No `Evento`, uma linha nova no `casts()`; na `Inscricao`, o campo novo no `$fillable`:

app/Models/Evento.php e app/Models/Inscricao.php (alterações)

```php
            'user_id' => 'integer',

    protected $fillable = ['user_id', 'nome', 'email', 'cpf', 'status'];
```

O *cast* garante que o `user_id` do evento sai no JSON sempre como número, e não como texto, seja qual for o banco. Isso importa para quem consome a API: o front-end vai comparar `evento.user_id === user.id`, e no JavaScript `"1" === 1` é falso. O `user_id` entra no `$fillable` da inscrição porque o controller vai gravá-lo junto com os outros campos. Isso não abre brecha: o que chega ao `create` é o `$dados` do `validate()`, que nunca traz o `user_id` do corpo da requisição. No evento ele não precisa entrar, como veremos a seguir.

PASSO 8.3 — CRIAR EM NOME DE QUEM ESTÁ LOGADO

No `store` do `EventoController`, troque `Evento::create($dados)` pela versão que parte do usuário do token. No `store` do `InscricaoController`, acrescente a linha do `user_id` logo acima da do `status`:

EventoController e InscricaoController (alterações no store)

```php
        $evento = $request->user()->eventos()->create($dados);

        $dados['user_id'] = $request->user()->id;
```

Teste criando um evento como Paula. Para ver que ninguém escolhe o dono pelo corpo da requisição, envie também um `user_id` que não é o dela:

POST {{base_url}}/eventos 201 Created

```
{
    "titulo": "Hackathon StartUB",
    "local": "Bloco B",
    "data": "2026-12-20 08:00",
    "vagas": 50,
    "valor": 0,
    "user_id": 2
}
```

RESPOSTA

```
{
    "titulo": "Hackathon StartUB",
    "local": "Bloco B",
    "data": "2026-12-20 08:00",
    "vagas": 50,
    "valor": "0.00",
    "user_id": 1,
    "updated_at": "2026-11-12T16:18:52.000000Z",
    "created_at": "2026-11-12T16:18:52.000000Z",
    "id": 3
}
```

O evento 3 é da Paula (`"user_id": 1`), apesar do `2` enviado. O `id` 3 voltou a ser usado porque o evento 3 do Passo 5.4 foi apagado: no SQLite, o próximo `id` é o maior existente mais um. O `validate()` descartou o campo, que não tem regra, e o `$request->user()->eventos()->create()` preencheu o `user_id` com o dono do token, do mesmo jeito que `$evento->inscricoes()->create()` preenche o `evento_id`. Esta é a regra de ouro da autorização: **o dono vem sempre do token, nunca do corpo da requisição.** Se viesse do corpo, bastaria escrever o id de outra pessoa para criar algo em nome dela.

PASSO 8.4 — AS REGRAS NUMA POLICY

```
PS> php artisan make:policy EventoPolicy
PS> php artisan make:policy InscricaoPolicy
```

TERMINAL

```
   INFO  Policy [app/Policies/EventoPolicy.php] created successfully.

   INFO  Policy [app/Policies/InscricaoPolicy.php] created successfully.
```

Substitua o conteúdo dos dois arquivos criados em `app/Policies`:

app/Policies/EventoPolicy.php

```php
<?php

namespace App\Policies;

use App\Models\Evento;
use App\Models\User;

class EventoPolicy
{
    public function update(User $user, Evento $evento): bool
    {
        return (int) $evento->user_id === $user->id;
    }

    public function delete(User $user, Evento $evento): bool
    {
        return $this->update($user, $evento);
    }

    public function viewInscricoes(User $user, Evento $evento): bool
    {
        return $this->update($user, $evento);
    }
}
```

app/Policies/InscricaoPolicy.php

```php
<?php

namespace App\Policies;

use App\Models\Inscricao;
use App\Models\User;

class InscricaoPolicy
{
    public function view(User $user, Inscricao $inscricao): bool
    {
        return (int) $inscricao->user_id === $user->id;
    }
}
```

Uma policy tem um método por ação. Cada método recebe o usuário logado e o registro, e responde `true` (pode) ou `false` (não pode). No evento, a regra é uma só, “é o dono?”, escrita no `update` e reaproveitada pelo `delete` e pelo `viewInscricoes`, o nome que demos à ação de ver os inscritos. Não há método para ver o evento: a leitura é livre. O `(int)` faz a comparação estrita funcionar mesmo que o valor venha do banco como texto, e um `user_id` nulo vira `0`, que não é o id de ninguém: registro sem dono não é de ninguém. Não é preciso registrar as policies em

lugar nenhum: o Laravel as encontra pelo nome, porque `App\Models\Evento` combina com `App\Policies\EventoPolicy`.

PASSO 8.5 — O CONTROLLER PEDE AUTORIZAÇÃO

Falta consultar a policy antes de cada ação restrita, com `Gate::authorize`. Nos dois controllers, acrescente o `use` no topo e uma linha no início de cada método restrito; o resto do código não muda:

app/Http/Controllers/EventoController.php (alterações)

```php
use Illuminate\Support\Facades\Gate;

    public function update(Request $request, Evento $evento)
    {
        Gate::authorize('update', $evento);
        // ... validação e update, como antes ...

    public function destroy(Evento $evento)
    {
        Gate::authorize('delete', $evento);
        // ... delete, como antes ...
```

app/Http/Controllers/InscricaoController.php (alterações)

```php
use Illuminate\Support\Facades\Gate;

    public function index(Evento $evento)
    {
        Gate::authorize('viewInscricoes', $evento);
        // ... listagem, como antes ...

    public function show(Inscricao $inscricao)
    {
        Gate::authorize('view', $inscricao);
        // ... load('evento'), como antes ...
```

O `index` e o `show` do `EventoController`, e o `store` das inscrições, ficam sem `Gate`: são as ações livres da vitrine.

`Gate::authorize('update', $evento)` descobre, pela classe do `$evento`, qual policy usar, chama o método `update` com o usuário logado e, se a resposta for `false`, interrompe a ação com `403`, do mesmo jeito que o `validate()` interrompe com `422`. No `update`, a autorização vem **antes** da validação: quem não é dono recebe `403` mesmo enviando dados inválidos, e não fica sabendo nada sobre as regras dos campos. O Quadro 9.1 resume quem pode o quê.

QUADRO 9.1 — REGRAS DE ACESSO DA API

Ação	Quem pode	Os outros
`POST /register`, `POST /login`	qualquer pessoa	—
`GET /eventos`, `GET /eventos/{id}`	qualquer conta logada (é a vitrine)	—
`POST /eventos`	qualquer conta logada, que vira dona do evento	—
`PATCH` e `DELETE /eventos/{id}`	só o dono do evento	`403`
`POST /eventos/{id}/inscricoes`	qualquer conta logada, que vira dona da inscrição	—
`GET /eventos/{id}/inscricoes`	só o dono do evento	`403`
`GET /inscricoes/{id}`	só quem fez a inscrição	`403`
qualquer rota protegida, sem token ou com token inválido	—	`401`

O `apiResource` registra `PUT` e `PATCH` para a alteração, e os dois passam pela mesma regra. Nos exemplos, usamos `PATCH`.

PASSO 8.6 — UMA SEGUNDA CONTA

Para testar as regras, precisamos de outra pessoa. Na coleção, crie uma pasta `Dono`. Nela, duplique *Auth — cadastrar* com os dados abaixo. O script da aba **Post-response** vem junto na cópia: depois do cadastro, o token do ambiente passa a ser o da Bia, e todas as requisições seguintes saem em nome dela. Duplique também *Auth — login* com o e-mail da Bia, para voltar à conta dela quando precisar.

POST `{{base_url}}/register` **201 Created**

```
{
    "name": "Bia Castro",
    "email": "bia@example.com",
    "password": "segredo123",
    "password_confirmation": "segredo123"
}
```

A Bia é o usuário 2. Ela enxerga o evento da Paula, e o campo `user_id` mostra de quem ele é:

GET {{base_url}}/eventos/3 200 OK

RESPOSTA


```
{
    "id": 3,
    "titulo": "Hackathon StartUB",
    "descricao": null,
    "local": "Bloco B",
    "data": "2026-12-20 08:00",
    "vagas": 50,
    "valor": "0.00",
    "created_at": "2026-11-12T16:18:52.000000Z",
    "updated_at": "2026-11-12T16:18:52.000000Z",
    "user_id": 1
}
```


Mas não pode alterá-lo:

PATCH {{base_url}}/eventos/3 403 Forbidden

```
{ "vagas": 10 }
```

RESPOSTA


```
{
    "message": "This action is unauthorized.",
    "exception": "Symfony\\Component\\HttpKernel\\Exception\\AccessDeniedHttpException",
    "file": "…",
    "line": 777,
    "trace": [ … ]
}
```


Como no `404` do Passo 4.3, os campos além da `message` só aparecem porque o projeto está com `APP_DEBUG=true`; a pilha foi resumida. O `DELETE {{base_url}}/eventos/3` recebe a mesma resposta. Já se inscrever é livre:

POST {{base_url}}/eventos/3/inscricoes 201 Created

```
{ "nome": "Bia Castro", "email": "bia@example.com", "cpf": "71428793860" }
```

RESPOSTA


```
{
    "nome": "Bia Castro",
    "email": "bia@example.com",
    "cpf": "71428793860",
    "user_id": 2,
    "status": "confirmada",
    "evento_id": 3,
    "updated_at": "2026-11-12T16:18:53.000000Z",
    "created_at": "2026-11-12T16:18:53.000000Z",
    "id": 4
}
```

A inscrição 4 é da Bia (`"user_id": 2`), e `GET {{base_url}}/inscricoes/4` responde `200` para ela. A lista de inscritos do evento, não: `GET {{base_url}}/eventos/3/inscricoes` responde `403` , porque o evento é da Paula.

PASSO 8.7 — DE VOLTA À DONA

Rode *Auth — login*, que grava de novo o token da Paula, e peça a lista de inscritos do evento dela:

GET `{{base_url}}/eventos/3/inscricoes` **200 OK**

RESPOSTA

```
[
    {
        "id": 4,
        "evento_id": 3,
        "nome": "Bia Castro",
        "email": "bia@example.com",
        "cpf": "71428793860",
        "status": "confirmada",
        "created_at": "2026-11-12T16:18:53.000000Z",
        "updated_at": "2026-11-12T16:18:53.000000Z",
        "user_id": 2
    }
]
```

A inscrição da Bia, sozinha, a Paula não abre: ser dona do evento não a torna dona das inscrições dos outros.

GET `{{base_url}}/inscricoes/4` **403 Forbidden**

RESPOSTA

```
{
    "message": "This action is unauthorized.",
    "exception": "Symfony\\Component\\HttpKernel\\Exception\\AccessDeniedHttpException",
    "file": "…",
    "line": 777,
    "trace": [ … ]
}
```

Por último, tente alterar o evento 1, que veio do seeder:

PATCH `{{base_url}}/eventos/1` **403 Forbidden**

```
{ "vagas": 40 }
```

RESPOSTA

```
{
    "message": "This action is unauthorized.",
    "exception": "Symfony\\Component\\HttpKernel\\Exception\\AccessDeniedHttpException",
    "file": "…",
    "line": 777,
    "trace": [ … ]
}
```

Os eventos 1 e 2 têm `user_id` nulo: pela API, ninguém os altera nem lista os seus inscritos. O mesmo vale para as inscrições de Ana, Bruno e Carla. Para dar um dono a eles, seria preciso preencher a coluna direto no banco, ou num seeder.

POR QUE 403, E NÃO 404?

Uma alternativa seria responder `404` a quem não é dono, como se o registro não existisse. Com `403`, quem tenta descobre que existe um evento com aquele `id`. Aqui isso foi aceito porque os eventos já são públicos na vitrine: o `403` não revela nada novo e deixa a resposta honesta, o que ajuda quem consome a API a entender o erro. Quando a simples existência do registro é informação sensível, como uma inscrição de outra pessoa num sistema de saúde, o `404` costuma ser a escolha melhor.

E NO FRONT-END?

Todo evento traz o `user_id`. Depois do login, o front-end guarda o `id` que vem de `GET /api/user` e mostra os botões *Editar* e *Remover* só nos eventos em que `evento.user_id === user.id`. Esconder o botão é conforto para quem usa a tela, não segurança: quem decide é a API, que responde `403` a quem chamar a rota direto, sem passar pela tela.

CONFIRA ANTES DE SEGUIR

Logada como Bia, `PATCH {{base_url}}/eventos/3` responde `403`, e a inscrição no evento 3 é aceita com `"user_id": 2`. Logada como Paula, a lista de inscritos do evento 3 vem com um item, e `GET {{base_url}}/inscricoes/4` responde `403`. Se tudo der `403` para a Paula, confira se o evento foi criado depois do Passo 8.3 (os anteriores não têm dono) e se o último login foi o dela. O projeto de referência traz esse roteiro como teste automático, em `tests/Feature/EventoDonoTest.php`; os testes são assunto do próximo encontro.

PARA DISCUSSÃO EM SALA

1. Por que não basta esconder o botão *Remover* no front-end para quem não é dono?
2. A Paula é dona do evento 3, mas recebe `403` ao abrir a inscrição da Bia, embora a veja na lista de inscritos. A regra faz sentido? Como você a mudaria, e em qual arquivo?
3. Hoje, qualquer conta cria eventos. Como seria uma regra em que só organizadores criam eventos? Que coluna e que método de policy você acrescentaria?

10

SÍNTESE TÉCNICA

O mapa da API e os erros mais comuns

O Quadro 10.1 reúne, para cada situação que a API trata, quem a detecta e qual resposta ela gera. Ele resume a divisão de responsabilidades que construímos: o **middleware** barra quem não se identificou, a **policy** barra quem se identificou mas não tem permissão, o **roteamento** resolve o que não existe, a **validação** resolve o que está malformado e o **controller** resolve o que viola regras do negócio.

QUADRO 10.1 — SITUAÇÕES, RESPONSÁVEIS E RESPOSTAS

Situação	Quem detecta	Resposta
Recurso encontrado / criado / removido	Controller	`200` / `201` / `204`
Requisição sem token, ou com token revogado	Middleware `auth:sanctum`, antes do controller	`401`
Login com e-mail ou senha errados	`AuthController`	`422`
Conta logada que não é a dona (alterar ou remover evento, ver inscritos, ver inscrição alheia)	Policy, consultada por `Gate::authorize` no controller	`403`
Evento ou inscrição inexistente	Route model binding, antes do controller	`404`
Campo ausente, tipo errado, e-mail repetido	`$request->validate()`	`422` + `errors`
Evento lotado	Regra de negócio no controller	`409`
E-mail repetido que escapou da validação	Índice `unique` no banco	`500` (último recurso)

QUADRO 10.2 — ERROS FREQUENTES NESTE ENCONTRO

Sintoma	Causa e solução
Could not send request no Postman	O `php artisan serve` não está rodando, ou o ambiente *Workshop local* não está selecionado e `{{base_url}}` ficou vazio.
`404` em uma rota que existe	Faltou o `/api` na URL, ou o arquivo de rotas não foi salvo. Confira com `route:list`.
`MassAssignmentException`	O campo não está no `$fillable` do model.
`no such table: inscricaos`	Faltou o `protected $table = 'inscricoes'` no model.
`no such column`	A migration foi alterada depois de executada. Use `migrate:fresh --seed` e refaça o cadastro: os usuários também são apagados.
`401 Unauthenticated` depois do login	O token não está indo: a coleção não está em **Bearer Token** `{{token}}`, a requisição não herda a autorização, ou o script não gravou a variável. Faça login de novo.
`Route [login] not defined`	Faltou o `redirectGuestsTo` do Passo 7.4 em `bootstrap/app.php`.
`403 This action is unauthorized.`	A conta logada não é a dona do registro. Confira de quem é o token (`GET /api/user`) e o `user_id` do evento; os eventos do seeder não têm dono.
Eventos novos com `"user_id": null`	O `store` ainda usa `Evento::create`: troque por `$request->user()->eventos()->create` (Passo 8.3).
Resposta em HTML no lugar de JSON	Rota fora de `/api` ou falta do `Accept: application/json`.

PRÁTICA

11 Exercícios

ORIENTAÇÕES

Os exercícios ampliam a API construída hoje e estão em ordem crescente de dificuldade. Para cada um, crie no Postman as requisições que provam que funciona, **inclusive os casos de erro**. Se estiver sem tempo, priorize os exercícios 1, 2 e 3: eles serão úteis no próximo encontro. As rotas novas entram **dentro** do grupo `auth:sanctum` e seguem as regras de dono da Etapa 8; teste-as com as duas contas, Paula e Bia.

1 Busca por título

Faça `GET /api/eventos?busca=git` devolver apenas os eventos cujo título contém o texto informado. Sem o parâmetro, a lista continua completa.

Dica: `$request->query('busca')` lê o parâmetro; a consulta pode ser montada aos poucos com `Evento::query()`, `->when(...)` e `->where('titulo', 'like', "%{$busca}%")`. O método `index` precisa receber `Request $request`.

2 Quantos inscritos?

Inclua em cada evento da listagem o número de inscrições, num campo `inscricoes_count`.

Dica: troque `Evento::orderBy('data')` por `Evento::withCount('inscricoes')->orderBy('data')`. O Laravel faz a contagem na mesma consulta.

3 Cancelar uma inscrição

Crie `DELETE /api/inscricoes/{inscricao}`, que **não apaga** o registro: muda o status para `cancelada` e devolve a inscrição. Só quem fez a inscrição pode cancelá-la; as outras contas recebem `403`. Em seguida, ajuste a regra de lotação para que inscrições canceladas não ocupem vaga.

Dica: para a contagem, `$evento->inscricoes()->where('status', '!=', 'cancelada')->count()`. Para a permissão, acrescente à `InscricaoPolicy` um método `delete` com a mesma regra do `view`. Pense: por que manter o registro em vez de apagar?

4 Mensagens em português

Faça a validação da inscrição responder em português, por exemplo *"O e-mail já está inscrito neste evento."* e *"Informe o CPF com 11 dígitos, sem pontuação."*.

Dica: `validate()` aceita um segundo array, de mensagens, com chaves no formato `'campo.regra'`: `'email.unique' => '…'`, `'cpf.digits' => '…'`.

5 Testes automáticos no Postman

Na requisição de criar evento, abra a aba **Scripts → Post-response** e escreva testes que confiram o status `201` e a presença do campo `id`. Depois, rode a coleção inteira com o **Runner**.

Dica: `pm.test("criado", () => pm.response.to.have.status(201));` e `pm.expect(pm.response.json()).to.have.property("id");`

6 Evento que já aconteceu

Recuse inscrições em eventos cuja data já passou, com status `409` e uma mensagem adequada. Para testar, altere a data de um evento direto no SQLite Viewer ou crie-o pelo seeder.

Dica: como `data` tem cast de data, `$evento->data->isPast()` responde à pergunta.

Desafio Painel do evento

Crie `GET /api/eventos/{evento}/resumo`, que devolve um JSON com: o título, o total de vagas, quantas inscrições há em cada status, as vagas restantes e o valor já arrecadado (inscrições confirmadas vezes o valor do evento). Como mostra quem se inscreveu e quanto entrou, o resumo é só para o dono do evento.

1. Decida o formato do JSON antes de programar e escreva-o como exemplo na descrição da requisição no Postman.
2. Implemente num método novo do `EventoController`.
3. Garanta que um evento sem inscrições devolva zeros, e não `null`.
4. Proteja a rota com a regra que já existe na `EventoPolicy`: `viewInscricoes`.

Dica: `$evento->inscricoes()->selectRaw('status, count(*) as total')->groupBy('status')->pluck('total', 'status')` devolve as contagens por status de uma vez.

12

FECHAMENTO

Síntese, autoavaliação e próximos passos

SÍNTESE DO ENCONTRO

- Sistemas conversam por **HTTP**: método, URL, cabeçalhos e corpo na ida; status, cabeçalhos e corpo na volta. O corpo, em APIs, é **JSON**.
- Uma API **REST** se organiza em recursos com URLs próprias, e o **método** diz a operação.
- No Laravel, a **rota** liga a URL ao **controller**, que usa **models** para acessar o banco descrito pelas **migrations**.
- **Validação** barra dados malformados (`422`); **regras de negócio** barram operações que conflitam com o estado do sistema (`409`).
- **Autenticação por token**: o login devolve um token, o cliente o envia em `Authorization: Bearer` e o middleware `auth:sanctum` responde `401` a quem não o tem.
- **Autorização**: o dono vem do token (`$request->user()`), nunca do corpo; as regras ficam numa **Policy**, o controller as consulta com `Gate::authorize`, e quem não pode recebe `403`.
- Relacionamentos (`hasMany` / `belongsTo`) deixam o código falar a língua do domínio: *as inscrições do evento*.

AUTOAVALIAÇÃO

1. Qual a diferença entre os status `422` e `409`? Dê um exemplo da nossa API para cada um.
2. O que faz a lista `$fillable` e que ataque ela previne?
3. Por que `GET /api/eventos/99` responde `404` se não escrevemos nenhum `if` para isso?
4. Por que a regra de e-mail único existe na validação **e** no banco? Uma não bastaria?
5. O que o cliente precisa enviar para acessar `GET /api/eventos`, e o que acontece com esse acesso depois do logout?
6. A Bia tenta alterar o evento da Paula e recebe `403`; sem token, receberia `401`. O que cada código diz ao cliente, e o que ele deve fazer em cada caso?
7. Por que o `user_id` do evento é preenchido com `$request->user()`, e não lido do corpo da requisição?
8. (Discursiva) O evento 2 cobra uma taxa e a inscrição nasce `pendente`. Descreva, com as suas palavras, como você imagina que a API deveria descobrir que o pagamento foi feito, e quais os problemas de cada alternativa que você considerar.

Glossário

API — conjunto de operações que um sistema oferece a outros programas.

Endpoint — a combinação de método e URL que executa uma operação da API.

Recurso — entidade exposta pela API (evento, inscrição), identificada por uma URL.

Migration — estrutura de uma tabela escrita em código e versionada com o projeto.

Model (Eloquent) — classe que representa uma tabela e dá acesso aos seus registros.

Route model binding — conversão automática do `{id}` da URL no registro do banco.

Mass assignment — preencher vários campos de uma vez a partir de um array; controlado pelo `$fillable`.

Seeder — classe que insere dados de exemplo no banco.

Token de acesso — credencial entregue no login e enviada pelo cliente em `Authorization: Bearer`.

Middleware — filtro que roda antes do controller, como o `auth:sanctum`.

Autorização — decidir se quem está logado pode executar a ação pedida; a recusa é o `403`.

Policy — classe com as regras de autorização de um model, um método por ação.

Para o próximo encontro

A inscrição de Bruno está pendente, e a nossa API ainda não tem como cobrar nem como saber que alguém pagou. No próximo encontro, ela vai **chamar uma API externa**, a do Asaas, para gerar a cobrança, e vai **receber chamadas** dele, por **webhook**, quando o pagamento for confirmado. Traga o projeto de hoje funcionando, o e-mail e a senha do usuário que você cadastrou, e a **chave de API do sandbox do Asaas**, criada conforme o material prévio.

Referências

1. FIELDING, R. T. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado) — University of California, Irvine, 2000.
2. LARAVEL. **Laravel 13.x Documentation**: Routing; Controllers; Validation; Eloquent: Relationships; Sanctum; Authorization. Disponível em: https://laravel.com/docs/13.x. Acesso em: nov. 2026.
3. MDN WEB DOCS. **HTTP response status codes**. Disponível em: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status. Acesso em: nov. 2026.
4. POSTMAN. **Write scripts to test API response data**. Learning Center. Disponível em: https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/. Acesso em: nov. 2026.

Material de uso didático, elaborado para o workshop *Desenvolvimento de APIs REST e Integração de Sistemas com Laravel*.